Guia da Equipe – Sprint 2 (04/11 → 12/11) e Sprint 3 (13/11 → 03/12)

Scrum Master: Murilo Alves. Este guia diz **o que cada um faz, como desenvolver e como provar que está pronto**.

1. Papéis e responsabilidades

Membro	Papel	Responsável por (Sprint 2)	Entrega validada por
Murilo Alves	Scrum Master	Board Trello, cerimônias, impedimentos, commit/merge final, ensaio, consolidar PDF da Sprint 2	Board atualizado + PDF completo
João Vitor G. Valli	Product Owner	Priorizar backlog, aceitar/recusar cards, distribuição de RF/RNF por Sprint, atualizar caso de uso/sequência junto com Júlia	Aceite no card (comentário "Aceito")
Leonardo Quadros	Desenvolvedor	Autenticação do operador (login + proteção dos endpoints de separação, ruptura, validação), expiração de pedidos	Testes + demo do fluxo sem login falhando (401)
Guilherme Reis	Desenvolvedor	Testes automatizados (`unittest`), separar `front.html` em HTML/CSS/JS, responsividade do painel (RNF005)	`python3 -m unittest` verde + print do painel em tela pequena
Júlia Normando	Analista de Requisitos / Testes	Diagramas de **classes** e de **atividades**, sequência de UC005 e UC007, tabela RF/RNF por Sprint, plano e roteiro de testes manuais	Diagramas exportados (.drawio + .png) revisados pelo PO

Todos: presença obrigatória na apresentação de 12/11 e postagem **individual** no LEX.

2. Backlog Sprint 2 (cards do Trello)

ID	Card	Dono	Prioridade
S2-01	Commitar pendências e criar pasta `documentacao/`	Murilo	P0
S2-02	Diagrama de classes	Júlia	P0
S2-03	Diagrama de atividades (compra → separação → retirada; ruptura)	Júlia	P0
S2-04	Sequência UC005 e UC007 + atualizar casos de uso	Júlia + João	P0
S2-05	Tabela RF/RNF por Sprint no documento	João	P0
S2-06	Login do operador e proteção dos endpoints	Leonardo	P1
S2-07	Testes automatizados dos fluxos principais	Guilherme	P1
S2-08	Separar `front.html` e ajustar responsividade	Guilherme	P1
S2-09	RF004: notificação de QR (mock documentado)	Leonardo	P2
S2-10	Ensaio e PDF final da Sprint 2	Murilo	P0

3. Como desenvolver

Rodar o projeto

```
python3 backend/app.py          # API em http://127.0.0.1:5000
python3 -m http.server 3000     # front em http://localhost:3000/front.html
```

Fluxo de trabalho (Git)

1. `git checkout main && git pull`
2. Crie branch por card: `git checkout -b feat/s2-06-login-operador`
3. Commits pequenos, no padrão do repo: `feat(g2): ...`, `fix(g2): ...`, `docs(g2): ...`, `test(g2): ...`
4. `git push -u origin <branch>` e abra **Pull Request** para `main`, citando o ID do card.
5. Outro membro revisa (Leonardo ↔ Guilherme; docs: Júlia ↔ João). Murilo faz o merge.

Regras

- Nada de segredo no código: chave HMAC e credenciais via variável de ambiente.
- Não commitar `*.db` (já no `.gitignore`).
- Não alterar arquivo de outro card sem avisar no grupo (risco de conflito, principalmente `front.html` e `services.py`).
- Diagramas: salvar o fonte `.drawio` **e** o `.png` em `documentacao/diagramas/`.

4. Como validar o progresso (cada um valida o próprio card)

Definition of Done (todo card):

- [ ] Critério de aceite do card cumprido
- [ ] Código/diagrama no GitHub via PR revisado
- [ ] Teste ou evidência anexada ao card (print, saída do teste, vídeo curto)
- [ ] README/documento atualizado se o comportamento mudou
- [ ] PO deu aceite

Checklists por responsável

Leonardo (login do operador)

- [ ] `POST /api/orders/<id>/separate` sem credencial retorna 401/403
- [ ] Mesmo endpoint com credencial válida funciona
- [ ] Idem para `/rupture` e `/validate-pickup`
- [ ] Cliente continua criando pedido sem login de operador
- [ ] Credenciais/segredo fora do código

Guilherme (testes e front)

- [ ] Testes cobrem: compra OK, pagamento recusado, estoque insuficiente, separação com SKU errado, ruptura (status + estorno + estoque zerado), retirada por voucher, retirada por terceiro, busca manual CPF

- [ ] `python3 -m unittest discover` passa do zero com banco limpo
- [ ] Front separado carrega sem erros no console do navegador
- [ ] Painel utilizável em largura ~360px

Júlia (diagramas e testes manuais)

- [ ] Classes batem com `backend/services.py` e `database.py` (atributos reais)
- [ ] Atividades cobrem fluxo principal e os alternativos (recusa, ruptura, QR inválido, terceiro)
- [ ] Nomenclatura dos UC/RF igual à do documento da Sprint 1
- [ ] Roteiro de teste manual executado com resultado registrado (passou/falhou)

João (PO)

- [ ] Cada card concluído revisado contra o critério de aceite
- [ ] Tabela RF/RNF por Sprint coerente com o que será demonstrado

Murilo (SM)

- [ ] Board reflete a realidade (sem card "fantasma" em Andamento)
- [ ] Impedimentos com dono e prazo
- [ ] Demo roda do zero em máquina limpa

5. Cerimônias

Evento	Quando	Duração	Pauta
Planning	04–05/11	40 min	Confirmar cards, donos e estimativas
Daily	Diária	15 min (pode ser por mensagem)	Ontem, hoje, impedimentos
Checkpoint	09/11	30 min	O que falta para 12/11; cortar P2 se necessário
Ensaio	11/11	45 min	Demo ponta a ponta com todos falando
Review	12/11	–	Apresentação ao professor
Retrospectiva	13/11	30 min	O que manter, parar, melhorar

Roteiro de demo sugerido (12/11): compra → recusa de cartão → separação (login operador) → ruptura → nova compra → retirada com voucher → retirada por terceiro → diagramas e RF/RNF por Sprint.

6. Sprint 3 (resumo)

Item	Dono
Modo offline/cache do terminal (RNF004)	Leonardo
Leitura real de QR Code (dependência a justificar antes)	Guilherme
HTTPS/segredos, medição de tempo (RNF001)	Leonardo + Guilherme
Atualização final de todos os diagramas e RF/RNF	Júlia
Roteiro e gravação do vídeo (25/11–01/12), revisão 02/12	Murilo + todos
Aceite final	João